

Exercice Java – Interfaces, héritage, algorithmes et diagrammes UML

Objectifs pédagogiques

- Comprendre le **contrat d'interface**
 - Implémenter une interface dans une **classe dédiée (composition)**
 - Appliquer l'**héritage de classes**, y compris en profondeur
 - Utiliser des **structures algorithmiques** dans les méthodes (boucles, conditions)
 - Manipuler un **tableau d'objets polymorphes**
 - Réaliser des **diagrammes UML** :
 - **de classes**
 - **de séquence** détaillé
 - Rédiger une **documentation professionnelle à publier sur GitHub**
-

Contexte

Tu dois modéliser un mini système animalier, dans lequel certains animaux savent **manger**, d'autres **crier**, certains font les deux. Les comportements seront définis via des **interfaces**, puis **implémentés séparément** dans des classes dédiées (pattern de *composition*).

Tu manipuleras un tableau d'animaux, tu appliqueras leurs comportements, et tu représenteras le fonctionnement dans un **diagramme UML de classes** et un **diagramme de séquence dynamique**.

Spécifications techniques

♦ Interfaces

Mangeant

- `void manger();`

- `int calculerBesoinEnergie();`

Criant

- `void crier();`
- `boolean estEnColere();`

♦ **Classe abstraite**

Animal (abstraite)

- Attributs :
 - `String nom`
 - `int age`
- Méthodes :
 - `void afficherInfos();`

♦ **Hiérarchie d'héritage**

- `Chien extends Animal`
- `BergerAllemand extends Chien`
- `Chat extends Animal`
- `ChatDeGouttiere extends Chat`

♦ **Implémentation des comportements**

MangeantImpl

- Attributs :
 - `int appetit`
 - `int niveauEnergie`
- Méthodes :
 - `void manger()`
 - Boucle 3 bouchées
 - Condition : si `Math.random() < 0.3`, afficher "Hmm, pas terrible !"
 - `int calculerBesoinEnergie()`
 - Calcul basé sur âge et appétit

CriantImpl

- Attributs :
 - `int niveauStress`
- Méthodes :
 - `void crier()` : message spécifique selon stress
 - `boolean estEnColere()` : retourne `true` si stress > 5

Ce que tu dois faire

1. Implémentation

- Crée toutes les classes et interfaces mentionnées.
- Utilise la composition dans les classes comme `BergerAllemand` ou `ChatDeGouttiere` pour déléguer à `MangeantImpl` et `CriantImpl`.

2. Méthode `main`

Dans `main`, tu devras :

- Créer un **tableau d'Animal**
 - Ex : un `BergerAllemand`, un `ChatDeGouttiere`, etc.
- Afficher leurs infos
- Si l'animal est **mangeant**, appeler `manger()` et `calculerBesoinEnergie()`
- Si l'animal est **criant**, appeler `crier()` et afficher s'il est en colère

Utilise :

- `instanceof` pour tester les interfaces
- des **boucles** pour parcourir le tableau
- des **conditions** pour adapter le comportement

Diagrammes UML demandés

♦ Diagramme de classes

Tu dois représenter :

- les **relations d'héritage**
- les **interfaces implémentées**
- les **compositions**
- les **attributs principaux et méthodes**

Outils possibles : Draw.io, Lucidchart, StarUML, ou PlantUML.

♦ Diagramme de séquence UML

Représente l'appel suivant dans le `main` :

```
Animal rex = new BergerAllemand("Rex", 5, 30);  
((Mangeant) rex).manger();
```

Tu dois :

- Montrer chaque **appel de méthode**
 - Représenter :
 - la délégation vers `MangeantImpl`
 - la **boucle de bouchées** (loop)
 - la **condition aléatoire** (alt)
 - un éventuel appel à `calculerBesoinEnergie()`
 - Utiliser la syntaxe UML ou PlantUML (`loop`, `alt`, etc.)
-



Document à rendre

- ♦ **1. Code complet**
 - Organisé, lisible, avec commentaires
- ♦ **2. Diagrammes UML**
 - Diagramme de classes
 - Diagramme de séquence
- ♦ **3. Résumé méthodologique (format GitHub)**

Dans un fichier `README.md` :

- Décris **comment tu as abordé l'exercice**
- Comment tu as structuré ton code
- Ce que tu as appris
- Les problèmes rencontrés et comment tu les as résolus

- Insère tes **diagrammes UML en image ou PlantUML**

💡 But : obtenir un **document professionnel** à publier sur ton GitHub pour valoriser ton travail.

✅ Rendu attendu

- Projet Java fonctionnel
 - UML en image ou `.puml`
 - README pro clair et bien structuré
-

Besoin d'aide ?

Si tu bloques, fais d'abord un **plan de ce que tu veux faire**, puis construis petit à petit.
Reviens au contrat de l'interface pour chaque comportement à implémenter.

Exercice Java – Version progressive guidée

Étape 1 : Préparer les interfaces

Crée l'interface `Mangeant` avec deux méthodes :

```
void manger();  
int calculerBesoinEnergie();
```

-

Crée l'interface `Criant` avec deux méthodes :

```
void crier();  
boolean estEnColere();
```

-

Objectif : Comprendre le rôle d'une interface comme contrat.

Étape 2 : Créer la classe abstraite `Animal`

- Attributs : `String nom, int age`
- Constructeur pour initialiser les attributs
- Méthode `afficherInfos()` qui affiche le nom et l'âge

Objectif : Gérer le socle commun à tous les animaux.

Étape 3 : Implémenter les classes héritées

- Crée la classe `Chien` qui étend `Animal`
- Crée la classe `BergerAllemand` qui étend `Chien`
- Crée la classe `Chat` qui étend `Animal`
- Crée la classe `ChatDeGouttiere` qui étend `Chat`

Objectif : S'approprier l'héritage multi-niveaux.

Étape 4 : Créer les classes d'implémentation des comportements

- `MangeantImpl` :
 - Attributs : `appetit, niveauEnergie`
 - Implémente `Mangeant`
 - Dans `manger()`, boucle 3 fois, simule bouchées avec condition aléatoire
 - Dans `calculerBesoinEnergie()`, calcule besoin selon appétit et âge (paramètre à passer au constructeur)
- `CriantImpl` :
 - Attribut : `niveauStress`
 - Implémente `Criant`

- `crier()` affiche un message selon stress
- `estEnColere()` renvoie `true` si stress > 5

Objectif : Distinguer interface/contrat et implémentation réelle.

Étape 5 : Composer les comportements dans les animaux

- Dans `BergerAllemand` :
 - Attribut privé `MangeantImpl mangeantImpl`
 - Délégation des méthodes `manger()` et `calculerBesoinEnergie()` vers `mangeantImpl`
- Dans `ChatDeGouttiere` :
 - Attributs privés `MangeantImpl mangeantImpl` et `CriantImpl criantImpl`
 - Délégation des méthodes des interfaces vers ces instances

Objectif : Appliquer le pattern de composition pour séparer responsabilités.

Étape 6 : Écrire la méthode `main`

- Crée un tableau `Animal[]` avec des instances de `BergerAllemand` et `ChatDeGouttiere`
- Parcours le tableau avec une boucle `for`
- Pour chaque animal :
 - Affiche ses infos
 - Si `instanceof Mangeant`, appelle `manger()` et affiche énergie
 - Si `instanceof Criant`, appelle `crier()` et affiche état colère

Objectif : Travailler avec le polymorphisme et la gestion dynamique des comportements.

Étape 7 : Réaliser les diagrammes UML

- **Diagramme de classes** : montre la hiérarchie, les interfaces, les compositions
- **Diagramme de séquence** : représente l'appel à `manger()` sur un `BergerAllemand`, avec boucle et condition

Objectif : Visualiser la conception et le déroulement du programme.

Étape 8 : Rédiger le résumé méthodologique

Dans un fichier `README.md`, décris :

- Ta démarche pas à pas
- Comment tu as modélisé les interfaces et classes
- Ce que tu as appris sur la délégation et la composition
- Tes difficultés et comment tu les as surmontées
- Intègre les diagrammes UML
- Explique comment tu comptes publier ce projet sur GitHub

Objectif : Mettre en valeur ton travail dans un cadre professionnel.